

Dokumentacja Techniczna: Genetic Algorithm Password Cracker

Mateusz Bogacz-Drewniak
Paweł Kruk

1. Wstęp

Niniejszy dokument opisuje architekturę, technologie oraz logikę działania aplikacji webowej służącej do demonstracji algorytmu genetycznego. Celem aplikacji jest odnalezienie zadanego ciągu znaków (hasła) poprzez ewolucję populacji losowych rozwiązań.

2. Stos Technologiczny

Projekt został zrealizowany w architekturze klient-serwer.

Backend (Warstwa Logiczna i API)

- **Python 3.x:** Główny język programowania logiki biznesowej.
- **FastAPI:** Wysokowydajny framework webowy służący do wystawienia API REST. Obsługuje żądania HTTP z frontendu oraz walidację danych wejściowych.
- **PyGAD:** Biblioteka Python dedykowana algorytmom genetycznym. Odpowiada za zarządzanie cyklem życia populacji (selekcja, krzyżowanie, mutacja).
- **NumPy:** Biblioteka do obliczeń naukowych, wykorzystywana wewnętrznie przez PyGAD do operacji macierzowych na genach.
- **Pydantic:** Biblioteka służąca do definicji modeli danych i ich walidacji (np. sprawdzanie zakresów parametrów mutacji).
- **Uvicorn:** Serwer ASGI służący do uruchamiania aplikacji FastAPI.

Frontend (Warstwa Prezentacji)

- **Svelte:** Kompilator i framework JavaScript do budowy interfejsu użytkownika. Zapewnia wysoką wydajność poprzez brak wirtualnego DOM.
- **Tailwind CSS:** Framework CSS typu "utility-first", wykorzystany do stylizacji interfejsu (responsywność, układ graficzny).
- **Chart.js:** Biblioteka do wizualizacji danych, użyta do generowania wykresu postępu (wzrostu wartości fitness w kolejnych generacjach).
- **Vite:** Narzędzie budujące (build tool), zapewniające środowisko deweloperskie i optymalizację kodu produkcyjnego.

3. Opis Teoretyczny Algorytmu

Algorytm genetyczny w tym projekcie jest algorytmem heurystycznym przeszukującym przestrzeń rozwiązań (wszystkich możliwych kombinacji znaków) w celu znalezienia ciągu identycznego z ciągiem docelowym (target).

3.1. Podstawowe definicje

- **Gen:** Pojedynczy znak. Pula genów obejmuje znaki ASCII (32-127) oraz polskie znaki diakrytyczne.
- **Chromosom (Osobnik):** Ciąg znaków o długości równej długości hasła docelowego. Reprezentuje potencjalne rozwiązanie.
- **Populacja:** Zbiór osobników w danej iteracji (generacji).

3.2. Funkcja Przystosowania (Fitness Function)

Funkcja oceny (Fitness) określa jakość danego osobnika. W tym przypadku jest to **liczba znaków zgodnych z celem na odpowiednich pozycjach**.

Wzór:

Fitness = Liczba przypadków, gdzie Osobnik[i] == Cel[i]

- Maksymalna wartość fitness jest równa długości szukanego hasła.
- Algorytm dąży do maksymalizacji tej funkcji.

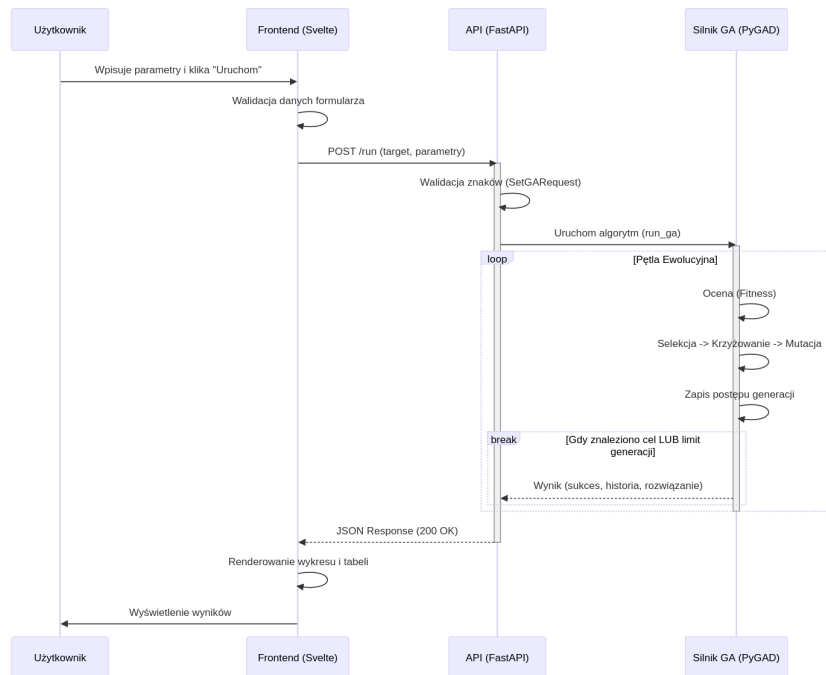
3.3. Cykl działania algorytmu

1. **Inicjalizacja:** Wygenerowanie populacji początkowej składającej się z losowych ciągów znaków.
 2. **Ewaluacja:** Obliczenie wartości fitness dla każdego osobnika w populacji.
 3. **Sprawdzenie warunku stopu:** Jeśli znaleziono osobnika o fitness równym długości celu lub osiągnięto limit generacji, algorytm kończy działanie.
 4. **Selekcja:** Wybór rodziców do reprodukcji metodą **turniejową** (Tournament Selection).
 5. **Krzyżowanie (Crossover):** Wymiana materiału genetycznego między rodzicami metodą **jednopunktową** (Single Point Crossover). Powstają nowi osobnicy (potomstwo).
 6. **Mutacja:** Losowa zmiana pojedynczych znaków w potomstwie na inne z puli dostępnych znaków (metoda random). Częstotliwość mutacji zależy od parametru `mutation_percent_genes`.
 7. **Elitaryzm:** Przeniesienie ustalonej liczby najlepszych rodziców (`keep_parents`) do nowej populacji bez zmian, aby nie utracić najlepszego dotychczasowego rozwiązania.
 8. **Powrót do punktu 2.**
-

4. Architektura Systemu i Diagramy

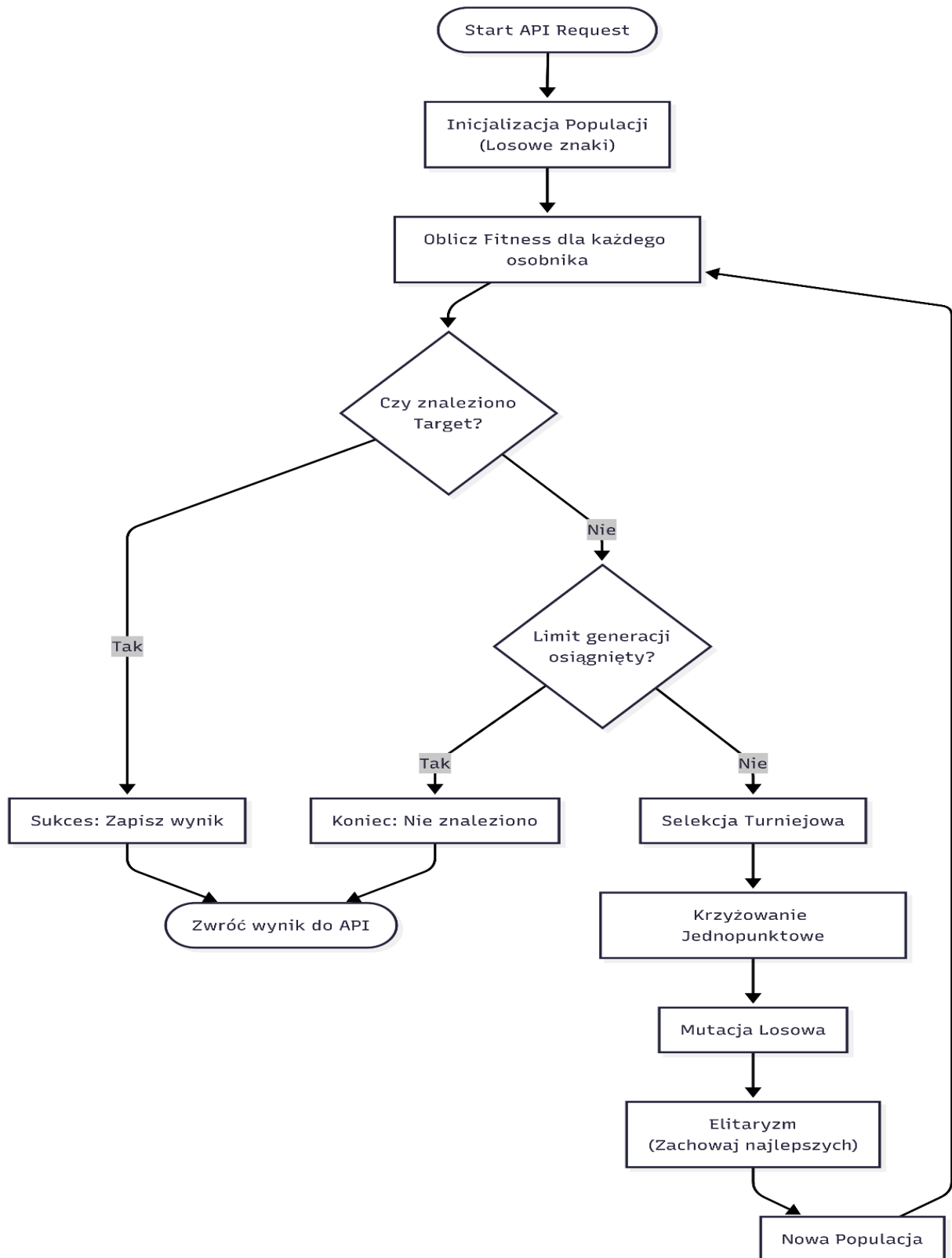
4.1. Diagram Sekwencji (Komunikacja Frontend - Backend)

Obrazuje przepływ danych od momentu kliknięcia przycisku "Uruchom" przez użytkownika.



4.2. Diagram Przepływu (Logika Algorytmu)

Wizualizacja procesu decyzyjnego wewnątrz modułu [ga.py](#)



5. Struktura Plików Projektu

- **main.py**: Punkt wejścia aplikacji backendowej. Konfiguruje serwer FastAPI, CORS oraz obsługuje endpoint `/run`.
- **ga.py**: Moduł zawierający logikę algorytmu genetycznego (konfiguracja PyGAD, funkcja fitness, callbacki zbierające dane o postępie).
- **set_ga_request.py**: Definicja modelu danych (Schema) dla żądania API. Określa typy zmiennych i ich ograniczenia (np. minimalna liczba generacji).
- **App.svelte**: Główny komponent frontendu. Zawiera:
 - Formularz konfiguracji parametrów.
 - Logikę komunikacji z API (fetch).
 - Komponenty wizualne (wykres Chart.js, tabela wyników).
- **tsconfig.app.json**: Konfiguracja kompilatora TypeScript dla aplikacji Svelte.

6. Endpointy API

POST /run

Uruchamia symulację algorytmu genetycznego.

Body (JSON):

- **target (string)**: Ciąg znaków do odgadnięcia.
- **max_generations (int)**: Maksymalna liczba iteracji.
- **parents_mating (int)**: Liczba rodziców biorących udział w reprodukcji.
- **sol_per_pop (int)**: Wielkość populacji.
- **keep_parents (int)**: Liczba najlepszych osobników przechodzących do kolejnej generacji bez zmian.
- **mutation_percent_genes (int)**: Procent genów podlegających mutacji.

Odpowiedź (JSON):

Zwraca obiekt zawierający m.in.: flagę sukcesu, ostateczne rozwiązanie, liczbę przeprowadzonych generacji oraz tablicę z historią postępu (dla wykresu).

7. Szczegółowy Opis Implementacji

W tej sekcji omówiono kluczowe fragmenty kodu odpowiedzialne za mechanikę algorytmu.

7.1. Przestrzeń poszukiwań (Moduł `ga.py`)

Algorytm operuje na zdefiniowanym zbiorze znaków (GENES). W skład tego zbioru wchodzi:

1. **Znaki ASCII**: Zakres kodów od 32 do 127 (cyfry, litery, znaki interpunkcyjne).
2. **Znaki Polskie**: Małe i wielkie litery (ą, ć, ę, ł, ń, ó, ś, ź, ż).

Zmienna `gene_space` w konfiguracji PyGAD jest listą list, co oznacza, że każdy gen (znak w hasle) może przyjąć dowolną wartość z indeksów tablicy `GENES`.

7.2. Funkcja Fitness (`fitness_func`)

Jest to serce algorytmu. Implementacja w Pythonie wygląda następująco:

```
def fitness_func(ga_instance, solution, solution_idx):  
    # Porównanie indeksów znaków w rozwiązaniu z indeksami znaków w celu  
    fitness = sum(1 for i in range(len(solution)) if int(solution[i]) == target_indices[i])  
    return fitness  
    # Przykładowa funkcja dopasowania dla algorytmu genetycznego
```

Algorytm nie porównuje bezpośrednio znaków (`char vs char`), lecz ich indeksy w tablicy `GENES`. Jeśli indeks znaku na pozycji `i` w osobniku zgadza się z indeksem znaku na pozycji `i` w hasle docelowym, `fitness` wzrasta o 1.

7.3. Callback generacji (`on_generation`)

Funkcja ta jest wywoływana po zakończeniu każdej ery (pokolenia). Odpowiada za:

1. Pobranie najlepszego rozwiązania w danej chwili.
2. Dekodowanie go z postaci liczbowej na tekst.
3. Zapisanie statystyk (numer generacji, aktualny ciąg, `fitness`) do listy `progress_list`.
4. **Warunek wczesnego stopu:** Jeśli `fitness == len(target)`, funkcja zwraca string `'stop'`, co natychmiast przerywa działanie algorytmu PyGAD, oszczędzając zasoby obliczeniowe.

8. Scenariusze Testowe i Wnioski

Poniżej przedstawiono przykładowe zachowanie algorytmu w zależności od parametrów wejściowych.

Scenariusz A: Krótkie hasło, standardowe parametry

- **Target:** "Hasło"
- **Długość:** 5 znaków
- **Populacja:** 100
- **Mutacja:** 10%
- **Wynik:** Algorytm zazwyczaj znajduje rozwiązanie w bardzo krótkim czasie (kilka-kilkanaście generacji), ponieważ prawdopodobieństwo wylosowania właściwego znaku jest wysokie przy małej przestrzeni poszukiwań.

Scenariusz B: Długie zdanie, mała populacja

- **Target:** "To jest trudne hasło do złamania"
- **Długość:** 33 znaki
- **Populacja:** 50
- **Mutacja:** 5%
- **Wynik:** Algorytm może utknąć w "lokalnym maksimum" (mieć większość liter dobrze, ale nie móc trafić w ostatnie kilka). Przy zbyt małej populacji różnorodność genetyczna szybko spada, a przy niskiej mutacji algorytm nie wprowadza nowych cech wystarczająco szybko. Wymaga to zwiększenia liczby generacji lub wielkości populacji.

Wnioski

1. **Złożoność:** Czas znalezienia rozwiązania nie rośnie liniowo wraz z długością hasła, lecz wykładniczo, choć mechanizmy selekcji i krzyżowania znacznie ten proces przyspieszają względem losowego przeszukiwania (Brute Force).
2. **Rola Mutacji:** Mutacja jest kluczowa dla zapobiegania przedwczesnej zbieżności algorytmu (sytuacja, w której cała populacja jest identyczna, ale nie jest rozwiązaniem idealnym).
3. **Elitaryzm:** Utrzymywanie `keep_parents > 0` gwarantuje, że wartość fitness nigdy nie spadnie w kolejnych generacjach (tzw. monotoniczny wzrost jakości rozwiązania).

10. Możliwości Rozwoju Projektu

W celu dalszego usprawnienia aplikacji można rozważyć następujące kroki:

1. **Obsługa Unicode:** Rozszerzenie puli genów o pełny zakres znaków Unicode (np. emoji, inne języki), co drastycznie zwiększy przestrzeń poszukiwań i trudność zadania.
2. **Wielowątkowość:** Wykorzystanie możliwości PyGAD do równoległego przetwarzania osobników, co przyspieszyłoby obliczenia dla bardzo dużych populacji.
3. **Wizualizacja "na żywo":** Zamiast czekać na zakończenie całego procesu, zastosowanie WebSocketów w FastAPI do przesyłania postępu generacji w czasie rzeczywistym do frontendu.
4. **Zapis wyników:** Dodanie bazy danych (np. SQLite) do przechowywania historii eksperymentów i porównywania skuteczności różnych parametrów.